Data Structure (CS100) | Unit I: Introduction & Linear Data Structures

DATA STRUCTURE

CS100

UNIT I — Introduction & Linear Data Structures

Part-I: Stack & Queue

BTechX

1. Introduction to Data & Data Structure

1.1 What is Data?

Data is a collection of facts, such as numbers, words, measurements, observations or descriptions of things. It can come in the form of text, observations, figures, images, numbers, graphs, or symbols.

- Examples: prices, weights, addresses, ages, names, temperatures, dates, distances

 Key Point: Processed data is called Information. Representation of data is called Structure.

1.2 What is a Data Structure?

A data structure is a specialized format for organizing, processing, retrieving and storing data. It is the logical or mathematical model of a particular organization of data.

- Provides an efficient way of storing and organizing data in the computer.
- Makes data usage efficient (quick access, easy traversal, easy searching).

1.3 Why Data Structures?

- Applications are becoming more complex and the amount of data is increasing every day.
- Problems with data searching, processing speed, multiple requests handling arise without proper structure.
- Data Structures support different methods to organize, manage, and store data efficiently.

Benefits of Data Structures

- Easier to traverse data items
- Accessing data becomes quick and efficient

- Searching of data becomes easy
- Provides efficient memory utilization

2. Types of Data Structures

2.1 Primitive vs Non-Primitive

Type	Description
Primitive	Predefined data types supported directly by the programming language. Can store only a single value. Examples: int, char, float, double, pointers.
Non-Primitive	Derived from primitive data types. Can hold multiple values in contiguous or random locations. Examples: Arrays, Stacks, Queues, Linked Lists.

2.2 Linear vs Non-Linear

Type	Description
Linear	Data stored in sequential or linear fashion. Each element is attached to its previous and next adjacent elements. Single run can traverse the structure. Types: Static and Dynamic. Examples: Array, Stack, Queue, Linked List.
Non-Linear	Data elements are not placed sequentially. Multiple paths between elements. Data elements connected to one or more elements. Examples: Tree, Graph.

2.3 Static vs Dynamic

Type	Description
Static	Fixed memory size. Easier to access elements. Example: Array.
Dynamic	Size is not fixed. Memory can be increased or decreased, providing more flexibility. Examples: Stack, Queue, Linked List.

3. Linear Data Structures — Overview

3.1 Array

- Collection of similar data types stored at contiguous memory locations.
- Uses index-based data structure to identify each element.
- Used to implement Stacks, Queues, Heaps, Hash tables, etc.

 Key Point: Data type of elements may be char, int, float, or double.

3.2 Stack (LIFO)

Stack is a linear data structure that follows the LIFO (Last In First Out) principle. Insertion (PUSH) and deletion (POP) operations are performed only from the top end.

3.3 Queue (FIFO)

Queue is a linear data structure that follows the FIFO (First In First Out) principle. Insertion at one end (REAR) and deletion at the other end (FRONT).

3.4 Linked List

Elements are stored non-contiguously. A pointer is used to link elements together. Head node stores the address of the first element.

3.5 Non-Linear — Tree & Graph

Structure	Description
Tree	Stores data on multiple levels. Top node is Root. Branches are Children. Represents hierarchical relationships. Consists of nodes connected by edges.
Graph	Pictorial representation of elements (vertices) connected by edges. Unlike trees, graphs can have cycles.

Real-Life Applications of Data Structures

- Stack — Browser back/forward buttons, Undo/Redo in text editors
- Queue — Printing in printers, CPU scheduling, ticket counters
- Tree — Social network graphs, file systems
- Graph — Google Maps (shortest path), social graphs

4. Array — In Depth

4.1 Types of Arrays

Type	Description
1D Array	Needs only one subscript to specify elements. Example: int arr[5];
2D Array	Needs two subscripts (rows and columns). Represents a matrix/table.
3D Array	Needs three subscripts. Used for complex data representations.

4.2 Operations on Array

Operation	Description
Traversal	Visiting every element of the array to print or process them.
Insertion	Adding an element at a particular index (beginning, end, or any position).
Deletion	Removing an element from a particular index.
Search	Finding an element using the given index or value.
Update	Replacing element at a particular index with a new value.

4.3 Insertion Algorithm

 Note: Array length(N), Element (ITEM) to insert, Position to insert (K)

```
1. Start
2. Set J = N-1
3. Set N = N+1
4. Repeat steps 5 and 6 while J >= K
5.    Set Arr[J+1] = Arr[J]
6.    Set J = J-1
7. Set Arr[K] = ITEM
8. Stop
```

4.4 Deletion Algorithm

 Note: Array length(N), Position to delete (K)

```
1. Start
2. Set ITEM = Arr[K]
3. Set J = K
4. Repeat steps 5 and 6 while J < N-1
5.    Set Arr[J] = Arr[J+1]
6.    Set J = J+1
7. Set N = N-1
8. Stop
```

4.5 Search Algorithm

 Note: Array length(N), Element (ITEM) to search

```
1. Start
2. Set J = 0
3. Repeat steps 4 and 5 while J < N
```

```
4.   If Arr[J] == ITEM Then Goto Step 6
5.   Set J = J+1
6. Print J, ITEM (found at position J)
7. Stop
```

4.6 Update Algorithm

 Note: Array length(N), Position K, New value NEW_ITEM

```
1. Start
2. Set Arr[K] = NEW_ITEM
3. Stop
```

4.7 Advantages & Disadvantages

Advantages	Disadvantages
Easier to declare and use	Number of elements must be predefined
Stores multiple elements of same type with same name	Array size cannot be changed after declaration
2D array represents tabular data efficiently	Allocating more memory than required leads to memory wastage

4.8 Address Calculations in 2D Array

Order	Formula
Row-Major Order	Address(A[i][j]) = B + (i*c + j) * w where B=base address, c=column size, w=element size
Column-Major Order	Address(A[i][j]) = B + (j*r + i) * w where B=base address, r=row size, w=element size

5. Stack — Complete Notes

5.1 Fundamentals of Stack

A stack is a linear data structure that follows the principle of Last In First Out (LIFO). Insertion (PUSH) and deletion (POP) operations are performed only from the top of the stack.

 Key Point: Think of a stack of plates — you always add and remove from the top!

5.2 Operations on Stack

Operation	Description
Push()	Add an element to the top of a stack
Pop()	Remove an element from the top of a stack
Peep() / Peek()	Get the value of the top element without removing it
IsEmpty()	Check if the stack is empty
IsFull()	Check if the stack is full
top()	Returns the top element of the stack
size()	Returns the size of the stack

5.3 PUSH Operation Algorithm

 Note: Input: STACK (linear array), MAX_SIZE. Initial: TOP = -1

```
PUSH (STACK, TOP, MAX_SIZE, ITEM)

Step 1: If TOP = MAX_SIZE - 1, then
           Print "Stack Overflow" and exit.
Step 2: Set TOP = TOP + 1
Step 3: Set STACK[TOP] = ITEM
Step 4: Exit
```

5.4 POP Operation Algorithm

 Note: Input: STACK (linear array), MAX_SIZE. Initial: TOP = -1

```
POP (STACK, TOP, MAX_SIZE, ITEM)

Step 1: If TOP = -1, then
           Print "Stack Underflow" and exit.
Step 2: Set ITEM = STACK[TOP]
Step 3: Set TOP = TOP - 1
Step 4: Exit
```

5.5 PEEP (Peek) Operation Algorithm

 Note: Returns the top element without removing it

```
PEEP (STACK, TOP, MAX_SIZE)

Step 1: If TOP = -1, then
```

```
            Print "Stack is Empty" and exit.
Step 2: Return STACK[TOP]
Step 3: Exit
```

5.6 Applications of Stack

Applications of Stack

- Browser history — back button saves previously visited URLs in a stack
- Undo/Redo operations in text editors
- Function calls and recursion (activation records)
- Expression evaluation (Postfix, Prefix)
- Conversion of expressions (Infix to Postfix / Prefix)
- Balanced Parentheses checking
- Tower of Hanoi
- To reverse a word or string

6. Recursion & Expression Conversion

6.1 Function Calls and Recursion

When a function calls itself directly or indirectly, it is called recursion. Every time a function is called, a structure called an activation record is created in memory, containing all local variables and parameters.

Example — Factorial using recursion:

```
long factorial(int n) {
    if (n > 1)
        return (n * factorial(n-1));
    else
        return 1;
}
```

 Key Point: Each function call creates a new stack frame (activation record). When the function returns, its frame is popped off the stack.

6.2 Arithmetic Expression Notations

Notation	Description	Example
Infix	Operator written between operands. Human-readable but needs precedence rules.	A + B
Prefix (Polish)	Operator written before operands. Machine-friendly, no parentheses needed.	+ A B
Postfix (Reverse Polish)	Operator written after operands. Used in compilers and calculators.	A B +

 Key Point: Prefix and Postfix notations are computationally efficient — they do not require parentheses or operator precedence tracking.

6.3 Operator Precedence & Associativity

Priority	Operator	Associativity
Highest	^ (Exponentiation)	Right to Left
High	*, / (Multiply, Divide)	Left to Right
Low	+, - (Add, Subtract)	Left to Right
Lowest	( ) Parentheses	Left to Right (override all)

6.4 Infix to Postfix Conversion (Manual Method)

Steps: Parenthesize from left to right. Sub-expressions converted to postfix are treated as single operands. Remove parentheses at the end.

Example: (A+B)*C/D+E^F/G

```
(A+B)*C/D+E^F/G
→ (AB+)*C/D+(EF^)/G         [() and ^ handled first]
→ ((AB+)C*)/D+(EF^)/G       [* left to right]
→ (((AB+)C*)D/)+(EF^)/G     [/ left to right]
→ (((AB+)C*)D/)+((EF^)G/)   [/ right side]
→ (((AB+)C*)D/)((EF^)G/)+   [+ last]

Postfix: AB+C*D/EF^G/+
```

6.5 Infix to Postfix Using Stack Algorithm

 Note: X = infix expression, Y = postfix output

```
1. Push '(' onto Stack, and add ')' to the end of X.
2. Scan X from left to right, repeat steps 3-6 until Stack is empty.
3. If operand encountered  → add it to Y.
4. If '(' encountered      → push it onto Stack.
5. If operator encountered →
   a. Pop from Stack and add to Y all operators with same/higher
precedence.
   b. Push current operator onto Stack.
6. If ')' encountered      →
   a. Pop from Stack and add to Y until '(' encountered.
   b. Remove '(' from Stack.
7. Exit
```

6.6 Infix to Postfix — Worked Example

Convert: A*(B+C+D)

Symbol Scanned	Stack	Postfix Expression
(	(	
A	(	A
*	(*	A
(	(*(	A
B	(*(	AB
+	(*(+	AB
C	(*(+	ABC
+	(*(+	ABC+ ← pop + (same precedence)
D	(*(+	ABC+D
)	(*	ABC+D+ ← pop until (
)		ABC+D+* ← pop until (

Result Postfix: ABC+D+*

6.7 Infix to Prefix Conversion

Steps:

- Step 1: Reverse the infix expression (swap '(' with ')' and vice versa)
- Step 2: Apply Infix-to-Postfix algorithm on the reversed expression
- Step 3: Reverse the resulting postfix expression to get the prefix

Example: a*(b+c+d)

```
Original infix:          a*(b+c+d)
Step 1 — Reverse:        )d+c+b(*a   → replace brackets: (d+c+b)*a
Step 2 — Postfix of (d+c+b)*a:  dc+b+a*
Step 3 — Reverse:        *a+b+cd

Prefix: *a+b+cd
```

7. Expression Evaluation using Stack

7.1 Postfix Expression Evaluation

 Note: Scan postfix expression from left to right

1. Scan expression from left to right.
2. If operand encountered → PUSH it onto stack.
3. If operator (#) encountered →
   a. Pop top element → A (top), Pop next → B (top-1).
   b. Perform operation: B # A
   c. Push result back to stack.
4. Final result is the remaining element in stack.

Example: 7 8 2 * 4 / +

```
Scan 7  → Stack: [7]
Scan 8  → Stack: [7, 8]
Scan 2  → Stack: [7, 8, 2]
Scan *  → Pop 2 and 8, compute 8*2=16 → Stack: [7, 16]
Scan 4  → Stack: [7, 16, 4]
Scan /  → Pop 4 and 16, compute 16/4=4 → Stack: [7, 4]
Scan +  → Pop 4 and 7, compute 7+4=11  → Stack: [11]

Result: 11
```

7.2 Prefix Expression Evaluation

 Note: Reverse prefix expression first, then scan left to right

1. Reverse the prefix expression.
2. Scan from left to right.
3. If operand encountered → PUSH it onto stack.
4. If operator (#) encountered →
   a. Pop top element → A (top), Pop next → B (top-1).
   b. Perform operation: A # B (Note: order reversed vs postfix)
   c. Push result back to stack.
5. Final result is the remaining element in stack.

 Note: For Postfix: operation is B # A (TOP-1 operator TOP). For Prefix (after reversing): operation is A # B (TOP operator TOP-1).

8. Queue — Complete Notes

8.1 Fundamentals of Queue

Queue is a linear data structure that follows the FIFO (First In First Out) principle. A queue enables insert operations at one end called REAR and delete operations at another end called FRONT.

 Key Point: Think of a queue of people waiting at a ticket counter — First come, first served!

8.2 Representation using Array

- Create an array of size n.
- Two variables: front and rear, both initialized to -1 (empty queue).
- REAR: index up to which elements are stored.
- FRONT: index of the first element of the array.

8.3 Types of Queue

Type	Description
Linear Queue	Simple FIFO queue. Limitation: memory wastage due to unutilized space after deletion.
Circular Queue	Last node is connected to the first node. Solves memory wastage problem. Uses modulo operation.
Double-Ended Queue (Deque)	Insertion and deletion from both ends. Hybrid of stack and queue.
Priority Queue	Elements are served based on priority, not order of insertion.

8.4 Insertion (Enqueue) Algorithm — Linear Queue

 Note: MAX = max size, FRONT and REAR initialized to -1

```
Step 1: If REAR = MAX - 1
           Print "OVERFLOW / QUEUE FULL" and EXIT
Step 2: If FRONT = -1 and REAR = -1
           Set FRONT = REAR = 0
        Else
           Set REAR = REAR + 1
Step 3: Set Q[REAR] = NUM
Step 4: EXIT
```

8.5 Deletion (Dequeue) Algorithm — Linear Queue

 Note: Delete from FRONT of queue

```
Step 1: If FRONT = -1 and REAR = -1
```

```
            Print "UNDERFLOW / QUEUE EMPTY" and EXIT
Step 2: Set NUM = Q[FRONT]
Step 3: If FRONT == REAR
            Set FRONT = REAR = -1
        Else
            Set FRONT = FRONT + 1
Step 4: EXIT
```

9. Circular Queue

9.1 Why Circular Queue?

Drawback of linear queue is wastage of memory due to unutilized memory space after deletions. A Circular Queue is an extended version of a linear queue where the last node is connected to the first node, making a circular link.

 Key Point: Circular Queue uses the Modulo (%) operation to wrap around: Rear = (Rear + 1) % MAX

9.2 Modulo Operation (Used in Circular Queue)

The modulus operator (%) returns the remainder when one number is divided by another.

```
Example 1:  7 % 5 = ?
7 = 5 * 1 + 2  →  Remainder = 2

Example 2: -7 % 5 = ?
Largest number <= -7 divisible by 5 is -10
Remainder = (-7) - (-10) = 3
```

9.3 Insertion Algorithm — Circular Queue

 Note: max = Maximum Queue Size

```
Step 1: Check for queue full:
   If (rear == max-1 AND front == 0) OR front == (rear+1) % max
   Then → OVERFLOW, Exit
Step 2: Check position of rear:
   If front == -1 AND Rear == -1 → set front = rear = 0
   Else if rear == max-1 AND front != 0 → set rear = 0
   Else → set rear = (rear+1) % max
Step 3: Q[rear] = Data
Step 4: Exit
```

9.4 Deletion Algorithm — Circular Queue

 Note: Delete from FRONT of circular queue

```
Step 1: If front == -1 → UNDERFLOW, Exit
Step 2: Data = Q[front]
Step 3: Update position:
   If front == Rear → set front = rear = -1
   Else if front = max-1 → set front = 0
   Else → set front = (front+1) % max
Step 4: Exit
```

Applications of Circular Queue

- Memory management
- Round Robin CPU Scheduling
- Traffic Light System
- Clock systems

10. Double-Ended Queue (Deque)

10.1 What is Deque?

Deque is a linear data structure where the insertion and deletion operations are performed from both ends. It is a hybrid linear structure that provides all the capabilities of stacks and queues in a single data structure.

10.2 Types of Deque

Type	Description
Input Restricted Deque	Insertion restricted at single end (REAR) only. Deletion allowed at BOTH ends (front and rear).
Output Restricted Deque	Deletion restricted at single end (FRONT) only. Insertion allowed at BOTH ends (front and rear).

10.3 Why Deque?

- Does not require the strict LIFO/FIFO ordering enforced by stack/queue.
- When random access is required but you also care about insertion and removal at both ends.
- Provides all capabilities of stacks and queues in a single data structure.

10.4 Operations on Deque

Operation	Description
Insertion at Rear	rear = rear + 1

Operation	Description
Insertion at Front	front = front - 1 (or wrap around using modulo)
Deletion at Front	front = front + 1
Deletion at Rear	rear = rear - 1 (when Rear reaches 0, set REAR = MAXSIZE - 1)

Applications of Deque

- Browser history: Recently visited URLs added to front; old URLs removed from back after a limit.
- Ticket purchasing line: Someone can re-join the front of the queue.
- Undo-Redo operations
- Palindrome checking

11. Quick Revision Summary

11.1 Key Terminologies

Term	Meaning
LIFO	Last In First Out — principle of Stack
FIFO	First In First Out — principle of Queue
PUSH	Insert element into Stack (from top)
POP	Delete element from Stack (from top)
Enqueue	Insert element into Queue (at REAR)
Dequeue	Delete element from Queue (at FRONT)
Overflow	Trying to insert into a full Stack/Queue
Underflow	Trying to delete from an empty Stack/Queue
Infix	Operator between operands: A+B
Prefix	Operator before operands: +AB
Postfix	Operator after operands: AB+
Deque	Double-Ended Queue — insert/delete at both ends
Modulo (%)	Returns remainder: 7 % 5 = 2; used in circular queue to wrap around

11.2 Practice Questions

Important Questions

- What is a Dequeue? Distinguish between LIFO and FIFO.

- Write PUSH and POP algorithms with examples.
- Convert (A+B)*C/D+E^F/G to Postfix and Prefix form.
- Evaluate postfix expression: 7 8 2 * 4 / +
- Evaluate postfix expression: 6 2 3 + - 3 8 2 / + *
- Evaluate prefix expression: + - * 3 2 / 8 4 1
- Explain circular queue with insertion and deletion example.
- Write an algorithm to insert/delete an element in a linear queue.
- Write algorithms to insert at beginning and end of an array of size 5.
- What are the applications of Stack and Queue?

12. References & Resources

12.1 Text Books

- "Data Structure: A Pseudo code approach with C" — Richard F. Gilberg & Behrouz A. Forouzan, Thomson Publication, 2nd Ed.
- "Data Structure in C" — A.M. Tanenbaum, Y. Langsam, M.J. Augenstein, PHI/Pearson, 7th Ed.

12.2 Reference Books

- "Data Structures with C" (Schaum's Series) — Seymour Lipschutz, Tata-McGraw-Hill, 1st Ed.
- "Fundamentals of Data Structure in C" — Horowitz, Sahani & Freed, Computer Science Press, 2nd Ed.
- "Data Structures Using C" — Reema Thareja, Oxford University Press, 2nd Ed.

12.3 Online Sources

- https://nptel.ac.in/courses/106104128 (Introduction to Programming in C)
- https://archive.nptel.ac.in/courses/106/102/106102064 (Data Structures and Algorithms)
- https://www.geeksforgeeks.org/data-structures/
- https://www.javatpoint.com/data-structure-introduction
- https://btechx.infinityfreeapp.com

— End of Unit I Notes —

BTechX